

5. Concurrency and Parallelism

Concurrency is when a computer does many different things *seemingly* at the same time. For example, on a computer with one CPU core, the operating system will rapidly change which program is running on the single processor. This interleaves execution of the programs, providing the illusion that the programs are running simultaneously.

Parallelism is *actually* doing many different things at the same time. Computers with multiple CPU cores can execute multiple programs simultaneously. Each CPU core runs the instructions of a separate program, allowing each program to make forward progress during the same instant.

Within a single program, concurrency is a tool that makes it easier for programmers to solve certain types of problems. Concurrent programs enable many distinct paths of execution to make forward progress in a way that seems to be both simultaneous and independent.

The key difference between parallelism and concurrency is *speedup*. When two distinct paths of execution in a program make forward progress in parallel, the time it takes to do the total work is cut in half; the speed of execution is faster by a factor of two. In contrast, concurrent programs may run thousands of separate paths of execution seemingly in parallel but provide no speedup for the total work.

Python makes it easy to write concurrent programs. Python can also be used to do parallel work through system calls, subprocesses, and C-extensions. But it can be very difficult to make concurrent Python code truly run in parallel. It's important to understand how to best utilize Python in these subtly different situations.

Item 36: Use subprocess to Manage Child Processes

Python has battle-hardened libraries for running and managing child processes. This makes Python a great language for gluing other tools together, such as command-line utilities. When existing shell scripts get complicated, as they often do over time, graduating them to a rewrite in Python is a natural choice for the sake of readability and maintainability.

Child processes started by Python are able to run in parallel, enabling you to use Python to consume all of the CPU cores of your machine and maximize the throughput of your programs. Although Python itself may be CPU bound (see [Item 37: “Use Threads for Blocking I/O, Avoid for Parallelism”](#)), it's easy to use Python to drive and coordinate CPU-intensive workloads.

Python has had many ways to run subprocesses over the years, including `popen`, `popen2`, and `os.exec*`. With the Python of today, the best and simplest choice for managing child processes is to use the `subprocess` built-in module.

Running a child process with `subprocess` is simple. Here, the `Popen` constructor starts the process. The `communicate` method reads the child process's output and waits for termination.

[Click here to view code image](#)

```
proc = subprocess.Popen(
    ['echo', 'Hello from the child!'],
    stdout=subprocess.PIPE)
out, err = proc.communicate()
print(out.decode('utf-8'))

>>>
Hello from the child!
```

Child processes will run independently from their parent process, the Python interpreter. Their status can be polled periodically while Python does other work.

[Click here to view code image](#)

```
proc = subprocess.Popen(['sleep', '0.3'])
while proc.poll() is None:
    print('Working...')
    # Some time-consuming work here
    # ...

print('Exit status', proc.poll())

>>>
Working...
Working...
Exit status 0
```

Decoupling the child process from the parent means that the parent process is free to run many child processes in parallel. You can do this by starting all the child processes together upfront.

[Click here to view code image](#)

```
def run_sleep(period):
    proc = subprocess.Popen(['sleep', str(period)])
    return proc

start = time()
procs = []
for _ in range(10):
    proc = run_sleep(0.1)
    procs.append(proc)
```

Later, you can wait for them to finish their I/O and terminate with the `communicate` method.

[Click here to view code image](#)

```
for proc in procs:
    proc.communicate()
end = time()
print('Finished in %.3f seconds' % (end - start))

>>>
Finished in 0.117 seconds
```

Note

If these processes ran in sequence, the total delay would be 1 second, not the ~0.1 second I measured.

You can also pipe data from your Python program into a subprocess and retrieve its output. This allows you to utilize other programs to do work in parallel. For example, say you want to use the `openssl` command-line tool to encrypt some data. Starting the child process with command-line arguments and I/O pipes is easy.

[Click here to view code image](#)

```
def run_openssl(data):
    env = os.environ.copy()
    env['password'] = b'\xe24U\n\xd0Ql3S\x11'
    proc = subprocess.Popen(
        ['openssl', 'enc', '-des3', '-pass', 'env:password'],
        env=env,
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE)
    proc.stdin.write(data)
    proc.stdin.flush() # Ensure the child gets input
    return proc
```

Here, I pipe random bytes into the encryption function, but in practice this would be user input, a file handle, a network socket, etc.:

```
procs = []
for _ in range(3):
    data = os.urandom(10)
    proc = run_openssl(data)
    procs.append(proc)
```

The child processes will run in parallel and consume their input. Here, I wait for them to finish and then retrieve their final output:

[Click here to view code image](#)

```
for proc in procs:
    out, err = proc.communicate()
    print(out[-10:])

>>>
b'o4,G\x91\x95\xfe\xa0\xaa\xb7'
b'\x0b\x01\\xb1\xb7\xfb\xb2C\xe1b'
b'ds\xc5\xf4;j\x1f\xd0c-'
```

You can also create chains of parallel processes just like UNIX pipes, connecting the output of one child process into the input of another, and so on. Here's a function that starts a child process that will cause the `md5` command-line tool to consume an input stream:

[Click here to view code image](#)

```
def run_md5(input_stdin):
    proc = subprocess.Popen(
        ['md5'],
        stdin=input_stdin,
        stdout=subprocess.PIPE)
```

```
return proc
```

Note

Python's `hashlib` built-in module provides the `md5` function, so running a subprocess like this isn't always necessary. The goal here is to demonstrate how subprocesses can pipe inputs and outputs.

Now, I can kick off a set of `openssl` processes to encrypt some data and another set of processes to `md5` hash the encrypted output.

[Click here to view code image](#)

```
input_procs = []
hash_procs = []
for _ in range(3):
    data = os.urandom(10)
    proc = run_openssl(data)
    input_procs.append(proc)
    hash_proc = run_md5(proc.stdout)
    hash_procs.append(hash_proc)
```

The I/O between the child processes will happen automatically once you get them started. All you need to do is wait for them to finish and print the final output.

[Click here to view code image](#)

```
for proc in input_procs:
    proc.communicate()
for proc in hash_procs:
    out, err = proc.communicate()
    print(out.strip())

>>>
b'7a1822875dcf9650a5a71e5e41e77bf3'
b'd41d8cd98f00b204e9800998ecf8427e'
b'1720f581cfdc448b6273048d42621100'
```

If you're worried about the child processes never finishing or somehow blocking on input or output pipes, then be sure to pass the `timeout` parameter to the `communicate` method. This will cause an exception to be raised if the child process hasn't responded within a time period, giving you a chance to terminate the misbehaving child.

[Click here to view code image](#)

```
proc = run_sleep(10)
try:
    proc.communicate(timeout=0.1)
except subprocess.TimeoutExpired:
    proc.terminate()
    proc.wait()

print('Exit status', proc.poll())

>>>
Exit status -15
```

Unfortunately, the `timeout` parameter is only available in Python 3.3 and later. In earlier versions of Python, you'll need to use the `select` built-in module on `proc.stdin`,

`proc.stdout`, and `proc.stderr` in order to enforce timeouts on I/O.

Things to Remember

- ◆ Use the `subprocess` module to run child processes and manage their input and output streams.
- ◆ Child processes run in parallel with the Python interpreter, enabling you to maximize your CPU usage.
- ◆ Use the `timeout` parameter with `communicate` to avoid deadlocks and hanging child processes.

Item 37: Use Threads for Blocking I/O, Avoid for Parallelism

The standard implementation of Python is called CPython. CPython runs a Python program in two steps. First, it parses and compiles the source text into bytecode. Then, it runs the bytecode using a stack-based interpreter. The bytecode interpreter has state that must be maintained and coherent while the Python program executes. Python enforces coherence with a mechanism called the *global interpreter lock* (GIL).

Essentially, the GIL is a mutual-exclusion lock (mutex) that prevents CPython from being affected by preemptive multithreading, where one thread takes control of a program by interrupting another thread. Such an interruption could corrupt the interpreter state if it comes at an unexpected time. The GIL prevents these interruptions and ensures that every bytecode instruction works correctly with the CPython implementation and its C-extension modules.

The GIL has an important negative side effect. With programs written in languages like C++ or Java, having multiple threads of execution means your program could utilize multiple CPU cores at the same time. Although Python supports multiple threads of execution, the GIL causes only one of them to make forward progress at a time. This means that when you reach for threads to do parallel computation and speed up your Python programs, you will be sorely disappointed.

For example, say you want to do something computationally intensive with Python. I'll use a naive number factorization algorithm as a proxy.

[Click here to view code image](#)

```
def factorize(number):
    for i in range(1, number + 1):
        if number % i == 0:
            yield i
```

Factoring a set of numbers in serial takes quite a long time.

[Click here to view code image](#)

```
numbers = [2139079, 1214759, 1516637, 1852285]
start = time()
for number in numbers:
    list(factorize(number))
```

```
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 1.040 seconds
```

Using multiple threads to do this computation would make sense in other languages because you could take advantage of all of the CPU cores of your computer. Let me try that in Python. Here, I define a Python thread for doing the same computation as before:

[Click here to view code image](#)

```
from threading import Thread

class FactorizeThread(Thread):
    def __init__(self, number):
        super().__init__()
        self.number = number

    def run(self):
        self.factors = list(factorize(self.number))
```

Then, I start a thread for factorizing each number in parallel.

[Click here to view code image](#)

```
start = time()
threads = []
for number in numbers:
    thread = FactorizeThread(number)
    thread.start()
    threads.append(thread)
```

Finally, I wait for all of the threads to finish.

[Click here to view code image](#)

```
for thread in threads:
    thread.join()
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 1.061 seconds
```

What's surprising is that this takes even longer than running `factorize` in serial. With one thread per number, you may expect less than a 4× speedup in other languages due to the overhead of creating threads and coordinating with them. You may expect only a 2× speedup on the dual-core machine I used to run this code. But you would never expect the performance of these threads to be worse when you have multiple CPUs to utilize. This demonstrates the effect of the GIL on programs running in the standard CPython interpreter.

There are ways to get CPython to utilize multiple cores, but it doesn't work with the standard `Thread` class (see [Item 41](#): “[Consider concurrent.futures for True Parallelism](#)”) and it can require substantial effort. Knowing these limitations you may wonder, why does Python support threads at all? There are two good reasons.

First, multiple threads make it easy for your program to seem like it's doing multiple things at the same time. Managing the juggling act of simultaneous tasks is difficult to

implement yourself (see [Item 40: “Consider Coroutines to Run Many Functions Concurrently”](#) for an example). With threads, you can leave it to Python to run your functions seemingly in parallel. This works because CPython ensures a level of fairness between Python threads of execution, even though only one of them makes forward progress at a time due to the GIL.

The second reason Python supports threads is to deal with blocking I/O, which happens when Python does certain types of system calls. System calls are how your Python program asks your computer’s operating system to interact with the external environment on your behalf. Blocking I/O includes things like reading and writing files, interacting with networks, communicating with devices like displays, etc. Threads help you handle blocking I/O by insulating your program from the time it takes for the operating system to respond to your requests.

For example, say you want to send a signal to a remote-controlled helicopter through a serial port. I’ll use a slow system call (`select`) as a proxy for this activity. This function asks the operating system to block for 0.1 second and then return control to my program, similar to what would happen when using a synchronous serial port.

[Click here to view code image](#)

```
import select

def slow_systemcall():
    select.select([], [], [], 0.1)
```

Running this system call in serial requires a linearly increasing amount of time.

[Click here to view code image](#)

```
start = time()
for _ in range(5):
    slow_systemcall()
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 0.503 seconds
```

The problem is that while the `slow_systemcall` function is running, my program can’t make any other progress. My program’s main thread of execution is blocked on the `select` system call. This situation is awful in practice. You need to be able to compute your helicopter’s next move while you’re sending it a signal, otherwise it’ll crash. When you find yourself needing to do blocking I/O and computation simultaneously, it’s time to consider moving your system calls to threads.

Here, I run multiple invocations of the `slow_systemcall` function in separate threads. This would allow you to communicate with multiple serial ports (and helicopters) at the same time, while leaving the main thread to do whatever computation is required.

[Click here to view code image](#)

```
start = time()
threads = []
for _ in range(5):
    thread = Thread(target=slow_systemcall)
    thread.start()
```



```
threads.append(thread)
```

With the threads started, here I do some work to calculate the next helicopter move before waiting for the system call threads to finish.

[Click here to view code image](#)

```
def compute_helicopter_location(index):
    # ...

for i in range(5):
    compute_helicopter_location(i)
for thread in threads:
    thread.join()
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 0.102 seconds
```

The parallel time is 5× less than the serial time. This shows that the system calls will all run in parallel from multiple Python threads even though they're limited by the GIL. The GIL prevents my Python code from running in parallel, but it has no negative effect on system calls. This works because Python threads release the GIL just before they make system calls and reacquire the GIL as soon as the system calls are done.

There are many other ways to deal with blocking I/O besides threads, such as the `asyncio` built-in module, and these alternatives have important benefits. But these options also require extra work in refactoring your code to fit a different model of execution (see [Item 40: “Consider Coroutines to Run Many Functions Concurrently”](#)). Using threads is the simplest way to do blocking I/O in parallel with minimal changes to your program.

Things to Remember

- ◆ Python threads can't run bytecode in parallel on multiple CPU cores because of the global interpreter lock (GIL).
- ◆ Python threads are still useful despite the GIL because they provide an easy way to do multiple things at seemingly the same time.
- ◆ Use Python threads to make multiple system calls in parallel. This allows you to do blocking I/O at the same time as computation.

Item 38: Use Lock to Prevent Data Races in Threads

After learning about the global interpreter lock (GIL) (see [Item 37: “Use Threads for Blocking I/O, Avoid for Parallelism”](#)), many new Python programmers assume they can forgo using mutual-exclusion locks (mutexes) in their code altogether. If the GIL is already preventing Python threads from running on multiple CPU cores in parallel, it must also act as a lock for a program's data structures, right? Some testing on types like lists and dictionaries may even show that this assumption appears to hold.

But beware, this is truly not the case. The GIL will not protect you. Although only one

Python thread runs at a time, a thread's operations on data structures can be interrupted between any two bytecode instructions in the Python interpreter. This is dangerous if you access the same objects from multiple threads simultaneously. The invariants of your data structures could be violated at practically any time because of these interruptions, leaving your program in a corrupted state.

For example, say you want to write a program that counts many things in parallel, like sampling light levels from a whole network of sensors. If you want to determine the total number of light samples over time, you can aggregate them with a new class.

[Click here to view code image](#)

```
class Counter(object):
    def __init__(self):
        self.count = 0

    def increment(self, offset):
        self.count += offset
```

Imagine that each sensor has its own worker thread because reading from the sensor requires blocking I/O. After each sensor measurement, the worker thread increments the counter up to a maximum number of desired readings.

[Click here to view code image](#)

```
def worker(sensor_index, how_many, counter):
    for _ in range(how_many):
        # Read from the sensor
        # ...
        counter.increment(1)
```

Here, I define a function that starts a worker thread for each sensor and waits for them all to finish their readings:

[Click here to view code image](#)

```
def run_threads(func, how_many, counter):
    threads = []
    for i in range(5):
        args = (i, how_many, counter)
        thread = Thread(target=func, args=args)
        threads.append(thread)
        thread.start()
    for thread in threads:
        thread.join()
```

Running five threads in parallel seems simple, and the outcome should be obvious.

[Click here to view code image](#)

```
how_many = 10**5
counter = Counter()
run_threads(worker, how_many, counter)
print('Counter should be %d, found %d' %
      (5 * how_many, counter.count))
```

```
>>>
Counter should be 500000, found 278328
```

But this result is way off! What happened here? How could something so simple go so wrong, especially since only one Python interpreter thread can run at a time?

The Python interpreter enforces fairness between all of the threads that are executing to ensure they get a roughly equal amount of processing time. To do this, Python will suspend a thread as it's running and will resume another thread in turn. The problem is that you don't know exactly when Python will suspend your threads. A thread can even be paused seemingly halfway through what looks like an atomic operation. That's what happened in this case.

The `Counter` object's `increment` method looks simple.

```
counter.count += offset
```

But the `+=` operator used on an object attribute actually instructs Python to do three separate operations behind the scenes. The statement above is equivalent to this:

[Click here to view code image](#)

```
value = getattr(counter, 'count')
result = value + offset
setattr(counter, 'count', result)
```

Python threads incrementing the counter can be suspended between any two of these operations. This is problematic if the way the operations interleave causes old versions of `value` to be assigned to the counter. Here's an example of bad interaction between two threads, A and B:

[Click here to view code image](#)

```
# Running in Thread A
value_a = getattr(counter, 'count')
# Context switch to Thread B
value_b = getattr(counter, 'count')
result_b = value_b + 1
setattr(counter, 'count', result_b)
# Context switch back to Thread A
result_a = value_a + 1
setattr(counter, 'count', result_a)
```

Thread A stomped on thread B, erasing all of its progress incrementing the counter. This is exactly what happened in the light sensor example above.

To prevent data races like these and other forms of data structure corruption, Python includes a robust set of tools in the `threading` built-in module. The simplest and most useful of them is the `Lock` class, a mutual-exclusion lock (mutex).

By using a lock, I can have the `Counter` class protect its current value against simultaneous access from multiple threads. Only one thread will be able to acquire the lock at a time. Here, I use a `with` statement to acquire and release the lock; this makes it easier to see which code is executing while the lock is held (see [Item 43: “Consider `contextlib` and `with` Statements for Reusable `try/finally` Behavior](#)” for details):

[Click here to view code image](#)

```
class LockingCounter(object):
    def __init__(self):
        self.lock = Lock()
        self.count = 0

    def increment(self, offset):
```

```
with self.lock:
    self.count += offset
```

Now I run the worker threads as before, but use a `LockingCounter` instead.

[Click here to view code image](#)

```
counter = LockingCounter()
run_threads(worker, how_many, counter)
print('Counter should be %d, found %d' %
      (5 * how_many, counter.count))
>>>
Counter should be 500000, found 500000
```

The result is exactly what I expect. The `LOCK` solved the problem.

Things to Remember

- ◆ Even though Python has a global interpreter lock, you're still responsible for protecting against data races between the threads in your programs.
- ◆ Your programs will corrupt their data structures if you allow multiple threads to modify the same objects without locks.
- ◆ The `Lock` class in the `threading` built-in module is Python's standard mutual exclusion lock implementation.

Item 39: Use Queue to Coordinate Work Between Threads

Python programs that do many things concurrently often need to coordinate their work. One of the most useful arrangements for concurrent work is a pipeline of functions.

A pipeline works like an assembly line used in manufacturing. Pipelines have many phases in serial with a specific function for each phase. New pieces of work are constantly added to the beginning of the pipeline. Each function can operate concurrently on the piece of work in its phase. The work moves forward as each function completes until there are no phases remaining. This approach is especially good for work that includes blocking I/O or subprocesses—activities that can easily be parallelized using Python (see [Item 37: “Use Threads for Blocking I/O, Avoid for Parallelism”](#)).

For example, say you want to build a system that will take a constant stream of images from your digital camera, resize them, and then add them to a photo gallery online. Such a program could be split into three phases of a pipeline. New images are retrieved in the first phase. The downloaded images are passed through the resize function in the second phase. The resized images are consumed by the upload function in the final phase.

Imagine you had already written Python functions that execute the phases: `download`, `resize`, `upload`. How do you assemble a pipeline to do the work concurrently?

The first thing you need is a way to hand off work between the pipeline phases. This can be modeled as a thread-safe producer-consumer queue (see [Item 38: “Use LOCK to Prevent Data Races in Threads”](#) to understand the importance of thread safety in Python; see [Item 46: “Use Built-in Algorithms and Data Structures”](#) for the `deque` class).

```
class MyQueue(object):
```

```
def __init__(self):
    self.items = deque()
    self.lock = Lock()
```

The producer, your digital camera, adds new images to the end of the list of pending items.

[Click here to view code image](#)

```
def put(self, item):
    with self.lock:
        self.items.append(item)
```

The consumer, the first phase of your processing pipeline, removes images from the front of the list of pending items.

[Click here to view code image](#)

```
def get(self):
    with self.lock:
        return self.items.popleft()
```

Here, I represent each phase of the pipeline as a Python thread that takes work from one queue like this, runs a function on it, and puts the result on another queue. I also track how many times the worker has checked for new input and how much work it's completed.

[Click here to view code image](#)

```
class Worker(Thread):
    def __init__(self, func, in_queue, out_queue):
        super().__init__()
        self.func = func
        self.in_queue = in_queue
        self.out_queue = out_queue
        self.polled_count = 0
        self.work_done = 0
```

The trickiest part is that the worker thread must properly handle the case where the input queue is empty because the previous phase hasn't completed its work yet. This happens where I catch the `IndexError` exception below. You can think of this as a holdup in the assembly line.

[Click here to view code image](#)

```
def run(self):
    while True:
        self.polled_count += 1
        try:
            item = self.in_queue.get()
        except IndexError:
            sleep(0.01) # No work to do
        else:
            result = self.func(item)
            self.out_queue.put(result)
            self.work_done += 1
```

Now I can connect the three phases together by creating the queues for their coordination points and the corresponding worker threads.

[Click here to view code image](#)

```
download_queue = MyQueue()
resize_queue = MyQueue()
```

```

upload_queue = MyQueue()
done_queue = MyQueue()
threads = [
    Worker(download, download_queue, resize_queue),
    Worker(resize, resize_queue, upload_queue),
    Worker(upload, upload_queue, done_queue),
]

```

I can start the threads and then inject a bunch of work into the first phase of the pipeline. Here, I use a plain `object` instance as a proxy for the real data required by the `download` function:

[Click here to view code image](#)

```

for thread in threads:
    thread.start()
for _ in range(1000):
    download_queue.put(object())

```

Now I wait for all of the items to be processed by the pipeline and end up in the `done_queue`.

[Click here to view code image](#)

```

while len(done_queue.items) < 1000:
    # Do something useful while waiting
    # ...

```

This runs properly, but there's an interesting side effect caused by the threads polling their input queues for new work. The tricky part, where I catch `IndexError` exceptions in the `run` method, executes a large number of times.

[Click here to view code image](#)

```

processed = len(done_queue.items)
polled = sum(t.polled_count for t in threads)
print('Processed', processed, 'items after polling',
      polled, 'times')

```

```
>>>
```

```
Processed 1000 items after polling 3030 times
```

When the worker functions vary in speeds, an earlier phase can prevent progress in later phases, backing up the pipeline. This causes later phases to starve and constantly check their input queues for new work in a tight loop. The outcome is that worker threads waste CPU time doing nothing useful (they're constantly raising and catching `IndexError` exceptions).

But that's just the beginning of what's wrong with this implementation. There are three more problems that you should also avoid. First, determining that all of the input work is complete requires yet another busy wait on the `done_queue`. Second, in `Worker` the `run` method will execute forever in its busy loop. There's no way to signal to a worker thread that it's time to exit.

Third, and worst of all, a backup in the pipeline can cause the program to crash arbitrarily. If the first phase makes rapid progress but the second phase makes slow progress, then the queue connecting the first phase to the second phase will constantly increase in size. The second phase won't be able to keep up. Given enough time and input data, the program

will eventually run out of memory and die.

The lesson here isn't that pipelines are bad; it's that it's hard to build a good producer-consumer queue yourself.

Queue to the Rescue

The `Queue` class from the `queue` built-in module provides all of the functionality you need to solve these problems.

`Queue` eliminates the busy waiting in the worker by making the `get` method block until new data is available. For example, here I start a thread that waits for some input data on a queue:

[Click here to view code image](#)

```
from queue import Queue
queue = Queue()

def consumer():
    print('Consumer waiting')
    queue.get()                # Runs after put() below
    print('Consumer done')

thread = Thread(target=consumer)
thread.start()
```

Even though the thread is running first, it won't finish until an item is put on the `Queue` instance and the `get` method has something to return.

[Click here to view code image](#)

```
print('Producer putting')
queue.put(object())          # Runs before get() above
thread.join()
print('Producer done')
>>>
Consumer waiting
Producer putting
Consumer done
Producer done
```

To solve the pipeline backup issue, the `Queue` class lets you specify the maximum amount of pending work you'll allow between two phases. This buffer size causes calls to `put` to block when the queue is already full. For example, here I define a thread that waits for a while before consuming a queue:

[Click here to view code image](#)

```
queue = Queue(1)             # Buffer size of 1

def consumer():
    time.sleep(0.1)           # Wait
    queue.get()               # Runs second
    print('Consumer got 1')
    queue.get()               # Runs fourth
    print('Consumer got 2')

thread = Thread(target=consumer)
```

```
thread.start()
```

The wait should allow the producer thread to put both objects on the queue before the consume thread ever calls get. But the Queue size is one. That means the producer adding items to the queue will have to wait for the consumer thread to call get at least once before the second call to put will stop blocking and add the second item to the queue.

[Click here to view code image](#)

```
queue.put(object())          # Runs first
print('Producer put 1')
queue.put(object())          # Runs third
print('Producer put 2')
thread.join()
print('Producer done')

>>>
Producer put 1
Consumer got 1
Producer put 2
Consumer got 2
Producer done
```

The Queue class can also track the progress of work using the task_done method. This lets you wait for a phase's input queue to drain and eliminates the need for polling the done_queue at the end of your pipeline. For example, here I define a consumer thread that calls task_done when it finishes working on an item.

[Click here to view code image](#)

```
in_queue = Queue()

def consumer():
    print('Consumer waiting')
    work = in_queue.get()      # Done second
    print('Consumer working')
    # Doing work
    # ...
    print('Consumer done')
    in_queue.task_done()      # Done third
```

```
Thread(target=consumer).start()
```

Now, the producer code doesn't have to join the consumer thread or poll. The producer can just wait for the in_queue to finish by calling join on the Queue instance. Even once it's empty, the in_queue won't be joinable until after task_done is called for every item that was ever enqueued.

[Click here to view code image](#)

```
in_queue.put(object())      # Done first
print('Producer waiting')
in_queue.join()              # Done fourth
print('Producer done')

>>>
Consumer waiting
Producer waiting
```



```
Consumer working
Consumer done
Producer done
```

I can put all of these behaviors together into a `Queue` subclass that also tells the worker thread when it should stop processing. Here, I define a `close` method that adds a special item to the queue that indicates there will be no more input items after it:

[Click here to view code image](#)

```
class ClosableQueue(Queue):
    SENTINEL = object()

    def close(self):
        self.put(self.SENTINEL)
```

Then, I define an iterator for the queue that looks for this special object and stops iteration when it's found. This `__iter__` method also calls `task_done` at appropriate times, letting me track the progress of work on the queue.

[Click here to view code image](#)

```
def __iter__(self):
    while True:
        item = self.get()
        try:
            if item is self.SENTINEL:
                return # Cause the thread to exit
            yield item
        finally:
            self.task_done()
```

Now, I can redefine my worker thread to rely on the behavior of the `ClosableQueue` class. The thread will exit once the for loop is exhausted.

[Click here to view code image](#)

```
class StoppableWorker(Thread):
    def __init__(self, func, in_queue, out_queue):
        # ...

    def run(self):
        for item in self.in_queue:
            result = self.func(item)
            self.out_queue.put(result)
```

Here, I re-create the set of worker threads using the new worker class:

[Click here to view code image](#)

```
download_queue = ClosableQueue()
# ...
threads = [
    StoppableWorker(download, download_queue, resize_queue),
    # ...
]
```

After running the worker threads like before, I also send the stop signal once all the input work has been injected by closing the input queue of the first phase.

[Click here to view code image](#)

```
for thread in threads:
```

```

        thread.start()
    for _ in range(1000):
        download_queue.put(object())
    download_queue.close()

```

Finally, I wait for the work to finish by joining each queue that connects the phases. Each time one phase is done, I signal the next phase to stop by closing its input queue. At the end, the `done_queue` contains all of the output objects as expected.

[Click here to view code image](#)

```

download_queue.join()
resize_queue.close()
resize_queue.join()
upload_queue.close()
upload_queue.join()
print(done_queue.qsize(), 'items finished')

```

```

>>>
1000 items finished

```

Things to Remember

- ◆ Pipelines are a great way to organize sequences of work that run concurrently using multiple Python threads.
- ◆ Be aware of the many problems in building concurrent pipelines: busy waiting, stopping workers, and memory explosion.
- ◆ The `Queue` class has all of the facilities you need to build robust pipelines: blocking operations, buffer sizes, and joining.

Item 40: Consider Coroutines to Run Many Functions Concurrently

Threads give Python programmers a way to run multiple functions seemingly at the same time (see [Item 37: “Use Threads for Blocking I/O, Avoid for Parallelism”](#)). But there are three big problems with threads:

- They require special tools to coordinate with each other safely (see [Item 38: “Use Lock to Prevent Data Races in Threads”](#) and [Item 39: “Use Queue to Coordinate Work Between Threads”](#)). This makes code that uses threads harder to reason about than procedural, single-threaded code. This complexity makes threaded code more difficult to extend and maintain over time.
- Threads require a lot of memory, about 8 MB per executing thread. On many computers, that amount of memory doesn’t matter for a dozen threads or so. But what if you want your program to run tens of thousands of functions “simultaneously”? These functions may correspond to user requests to a server, pixels on a screen, particles in a simulation, etc. Running a thread per unique activity just won’t work.
- Threads are costly to start. If you want to constantly be creating new concurrent functions and finishing them, the overhead of using threads becomes large and slows

everything down.

Python can work around all these issues with *coroutines*. Coroutines let you have many seemingly simultaneous functions in your Python programs. They're implemented as an extension to generators (see [Item 16: “Consider Generators Instead of Returning Lists”](#)). The cost of starting a generator coroutine is a function call. Once active, they each use less than 1 KB of memory until they're exhausted.

Coroutines work by enabling the code consuming a generator to `send` a value back into the generator function after each `yield` expression. The generator function receives the value passed to the `send` function as the result of the corresponding `yield` expression.

[Click here to view code image](#)

```
def my_coroutine():
    while True:
        received = yield
        print('Received:', received)

it = my_coroutine()
next(it)                # Prime the coroutine
it.send('First')
it.send('Second')

>>>
Received: First
Received: Second
```

The initial call to `next` is required to prepare the generator for receiving the first `send` by advancing it to the first `yield` expression. Together, `yield` and `send` provide generators with a standard way to vary their next yielded value in response to external input.

For example, say you want to implement a generator coroutine that yields the minimum value it's been sent so far. Here, the bare `yield` prepares the coroutine with the initial minimum value sent in from the outside. Then the generator repeatedly yields the new minimum in exchange for the next value to consider.

[Click here to view code image](#)

```
def minimize():
    current = yield
    while True:
        value = yield current
        current = min(value, current)
```

The code consuming the generator can run one step at a time and will output the minimum value seen after each input.

[Click here to view code image](#)

```
it = minimize()
next(it)                # Prime the generator
print(it.send(10))
print(it.send(4))
print(it.send(22))
print(it.send(-1))

>>>
```

```
10
4
4
-1
```

The generator function will seemingly run forever, making forward progress with each new call to `send`. Like threads, coroutines are independent functions that can consume inputs from their environment and produce resulting outputs. The difference is that coroutines pause at each `yield` expression in the generator function and resume after each call to `send` from the outside. This is the magical mechanism of coroutines.

This behavior allows the code consuming the generator to take action after each `yield` expression in the coroutine. The consuming code can use the generator's output values to call other functions and update data structures. Most importantly, it can advance other generator functions until their next `yield` expressions. By advancing many separate generators in lockstep, they will all seem to be running simultaneously, mimicking the concurrent behavior of Python threads.

The Game of Life

Let me demonstrate the simultaneous behavior of coroutines with an example. Say you want to use coroutines to implement Conway's Game of Life. The rules of the game are simple. You have a two-dimensional grid of an arbitrary size. Each cell in the grid can either be alive or empty.

```
ALIVE = '*'
EMPTY = '-'
```

The game progresses one tick of the clock at a time. At each tick, each cell counts how many of its neighboring eight cells are still alive. Based on its neighbor count, each cell decides if it will keep living, die, or regenerate. Here's an example of a 5×5 Game of Life grid after four generations with time going to the right. I'll explain the specific rules further below.

[Click here to view code image](#)

```
  0   |   1   |   2   |   3   |   4
-- | -- | -- | -- | --
-* - | -* - | -**- | -* - | --
-**- | -**- | -* - | -* - | -**-
-* - | -**- | -**- | -* - | --
-- | -- | -- | -- | --
```

I can model this game by representing each cell as a generator coroutine running in lockstep with all the others.

To implement this, first I need a way to retrieve the status of neighboring cells. I can do this with a coroutine named `count_neighbors` that works by yielding `Query` objects. The `Query` class I define myself. Its purpose is to provide the generator coroutine with a way to ask its surrounding environment for information.

[Click here to view code image](#)

```
Query = namedtuple('Query', ('y', 'x'))
```

The coroutine yields a `Query` for each neighbor. The result of each `yield` expression

will be the value `ALIVE` or `EMPTY`. That's the interface contract I've defined between the coroutine and its consuming code. The `count_neighbors` generator sees the neighbors' states and returns the count of living neighbors.

[Click here to view code image](#)

```
def count_neighbors(y, x):
    n_ = yield Query(y + 1, x + 0) # North
    ne = yield Query(y + 1, x + 1) # Northeast
    # Define e_, se, s_, sw, w_, nw ...
    # ...
    neighbor_states = [n_, ne, e_, se, s_, sw, w_, nw]
    count = 0
    for state in neighbor_states:
        if state == ALIVE:
            count += 1
    return count
```

I can drive the `count_neighbors` coroutine with fake data to test it. Here, I show how `Query` objects will be yielded for each neighbor. `count_neighbors` expects to receive cell states corresponding to each `Query` through the coroutine's `send` method. The final count is returned in the `StopIteration` exception that is raised when the generator is exhausted by the `return` statement.

[Click here to view code image](#)

```
it = count_neighbors(10, 5)
q1 = next(it) # Get the first query
print('First yield: ', q1)
q2 = it.send(ALIVE) # Send q1 state, get q2
print('Second yield:', q2)
q3 = it.send(ALIVE) # Send q2 state, get q3
# ...
try:
    count = it.send(EMPTY) # Send q8 state, retrieve count
except StopIteration as e:
    print('Count: ', e.value) # Value from return statement
>>>
First yield: Query(y=11, x=5)
Second yield: Query(y=11, x=6)
...
Count: 2
```

Now I need the ability to indicate that a cell will transition to a new state in response to the neighbor count that it found from `count_neighbors`. To do this, I define another coroutine called `step_cell`. This generator will indicate transitions in a cell's state by yielding `Transition` objects. This is another class that I define, just like the `Query` class.

[Click here to view code image](#)

```
Transition = namedtuple('Transition', ('y', 'x', 'state'))
```

The `step_cell` coroutine receives its coordinates in the grid as arguments. It yields a `Query` to get the initial state of those coordinates. It runs `count_neighbors` to inspect the cells around it. It runs the game logic to determine what state the cell should have for the next clock tick. Finally, it yields a `Transition` object to tell the environment the cell's next state.

[Click here to view code image](#)

```
def game_logic(state, neighbors):
    # ...

def step_cell(y, x):
    state = yield Query(y, x)
    neighbors = yield from count_neighbors(y, x)
    next_state = game_logic(state, neighbors)
    yield Transition(y, x, next_state)
```

Importantly, the call to `count_neighbors` uses the `yield from` expression. This expression allows Python to compose generator coroutines together, making it easy to reuse smaller pieces of functionality and build complex coroutines from simpler ones. When `count_neighbors` is exhausted, the final value it returns (with the `return` statement) will be passed to `step_cell` as the result of the `yield from` expression.

Now, I can finally define the simple game logic for Conway's Game of Life. There are only three rules.

[Click here to view code image](#)

```
def game_logic(state, neighbors):
    if state == ALIVE:
        if neighbors < 2:
            return EMPTY    # Die: Too few
        elif neighbors > 3:
            return EMPTY    # Die: Too many
    else:
        if neighbors == 3:
            return ALIVE    # Regenerate
    return state
```

I can drive the `step_cell` coroutine with fake data to test it.

[Click here to view code image](#)

```
it = step_cell(10, 5)
q0 = next(it)           # Initial location query
print('Me: ', q0)
q1 = it.send(ALIVE)     # Send my status, get neighbor query
print('Q1: ', q1)
# ...
t1 = it.send(EMPTY)     # Send for q8, get game decision
print('Outcome: ', t1)

>>>
Me:      Query(y=10, x=5)
Q1:      Query(y=11, x=5)
...
Outcome: Transition(y=10, x=5, state='-')
```

The goal of the game is to run this logic for a whole grid of cells in lockstep. To do this, I can further compose the `step_cell` coroutine into a `simulate` coroutine. This coroutine progresses the grid of cells forward by yielding from `step_cell` many times. After progressing every coordinate, it yields a `TICK` object to indicate that the current generation of cells have all transitioned.

[Click here to view code image](#)

```
TICK = object()

def simulate(height, width):
    while True:
        for y in range(height):
            for x in range(width):
                yield from step_cell(y, x)
            yield TICK
```

What's impressive about `simulate` is that it's completely disconnected from the surrounding environment. I still haven't defined how the grid is represented in Python objects, how `Query`, `Transition`, and `TICK` values are handled on the outside, nor how the game gets its initial state. But the logic is clear. Each cell will transition by running `step_cell`. Then the game clock will tick. This will continue forever, as long as the `simulate` coroutine is advanced.

This is the beauty of coroutines. They help you focus on the logic of what you're trying to accomplish. They decouple your code's instructions for the environment from the implementation that carries out your wishes. This enables you to run coroutines seemingly in parallel. This also allows you to improve the implementation of following those instructions over time without changing the coroutines.

Now, I want to run `simulate` in a real environment. To do that, I need to represent the state of each cell in the grid. Here, I define a class to contain the grid:

[Click here to view code image](#)

```
class Grid(object):
    def __init__(self, height, width):
        self.height = height
        self.width = width
        self.rows = []
        for _ in range(self.height):
            self.rows.append([EMPTY] * self.width)

    def __str__(self):
        # ...
```

The grid allows you to get and set the value of any coordinate. Coordinates that are out of bounds will wrap around, making the grid act like infinite looping space.

[Click here to view code image](#)

```
def query(self, y, x):
    return self.rows[y % self.height][x % self.width]

def assign(self, y, x, state):
    self.rows[y % self.height][x % self.width] = state
```

At last, I can define the function that interprets the values yielded from `simulate` and all of its interior coroutines. This function turns the instructions from the coroutines into interactions with the surrounding environment. It progresses the whole grid of cells forward a single step and then returns a new grid containing the next state.

[Click here to view code image](#)

```
def live_a_generation(grid, sim):
    progeny = Grid(grid.height, grid.width)
    item = next(sim)
```



```

while item is not TICK:
    if isinstance(item, Query):
        state = grid.query(item.y, item.x)
        item = sim.send(state)
    else: # Must be a Transition
        progeny.assign(item.y, item.x, item.state)
        item = next(sim)
return progeny

```

To see this function in action, I need to create a grid and set its initial state. Here, I make a classic shape called a glider.

```

grid = Grid(5, 9)
grid.assign(0, 3, ALIVE)
# ...
print(grid)

```

```

>>>
_*__
--*__
***__
---
---
```

Now I can progress this grid forward one generation at a time. You can see how the glider moves down and to the right on the grid based on the simple rules from the `game_logic` function.

[Click here to view code image](#)

```

class ColumnPrinter(object):
    # ...

columns = ColumnPrinter()
sim = simulate(grid.height, grid.width)
for i in range(5):
    columns.append(str(grid))
    grid = live_a_generation(grid, sim)

print(columns)

```

```

>>>
      0      |      1      |      2      |      3      |      4
_*__ | ___ | ___ | ___ | ___
--*__ | -*__ | --*__ | -*__ | --*__
***__ | **__ | -*__ | --** | --*_
--- | -*__ | -**__ | -**__ | -***
--- | ___ | ___ | ___ | ___
```

The best part about this approach is that I can change the `game_logic` function without having to update the code that surrounds it. I can change the rules or add larger spheres of influence with the existing mechanics of `Query`, `Transition`, and `TICK`. This demonstrates how coroutines enable the separation of concerns, which is an important design principle.

Coroutines in Python 2

Unfortunately, Python 2 is missing some of the syntactical sugar that makes coroutines so elegant in Python 3. There are two limitations. First, there is no `yield from` expression. That means that when you want to compose generator coroutines in Python 2, you need to include an additional loop at the delegation point.

[Click here to view code image](#)

```
# Python 2
def delegated():
    yield 1
    yield 2

def composed():
    yield 'A'
    for value in delegated(): # yield from in Python 3
        yield value
    yield 'B'

print list(composed())

>>>
['A', 1, 2, 'B']
```

The second limitation is that there is no support for the `return` statement in Python 2 generators. To get the same behavior that interacts correctly with `try/except/finally` blocks, you need to define your own exception type and raise it when you want to return a value.

[Click here to view code image](#)

```
# Python 2
class MyReturn(Exception):
    def __init__(self, value):
        self.value = value

def delegated():
    yield 1
    raise MyReturn(2) # return 2 in Python 3
    yield 'Not reached'

def composed():
    try:
        for value in delegated():
            yield value
    except MyReturn as e:
        output = e.value
    yield output * 4

print list(composed())

>>>
[1, 8]
```

Things to Remember

- ◆ Coroutines provide an efficient way to run tens of thousands of functions seemingly at the same time.
- ◆ Within a generator, the value of the `yield` expression will be whatever value was passed to the generator's `send` method from the exterior code.
- ◆ Coroutines give you a powerful tool for separating the core logic of your program from its interaction with the surrounding environment.
- ◆ Python 2 doesn't support `yield from` or returning values from generators.

Item 41: Consider `concurrent.futures` for True Parallelism

At some point in writing Python programs, you may hit the performance wall. Even after optimizing your code (see [Item 58: “Profile Before Optimizing”](#)), your program's execution may still be too slow for your needs. On modern computers that have an increasing number of CPU cores, it's reasonable to assume that one solution would be parallelism. What if you could split your code's computation into independent pieces of work that run simultaneously across multiple CPU cores?

Unfortunately, Python's global interpreter lock (GIL) prevents true parallelism in threads (see [Item 37: “Use Threads for Blocking I/O, Avoid for Parallelism”](#)), so that option is out. Another common suggestion is to rewrite your most performance-critical code as an extension module using the C language. C gets you closer to the bare metal and can run faster than Python, eliminating the need for parallelism. C-extensions can also start native threads that run in parallel and utilize multiple CPU cores. Python's API for C-extensions is well documented and a good choice for an escape hatch.

But rewriting your code in C has a high cost. Code that is short and understandable in Python can become verbose and complicated in C. Such a port requires extensive testing to ensure that the functionality is equivalent to the original Python code and that no bugs have been introduced. Sometimes it's worth it, which explains the large ecosystem of C-extension modules in the Python community that speed up things like text parsing, image compositing, and matrix math. There are even open source tools such as Cython (<http://cython.org/>) and Numba (<http://numba.pydata.org/>) that can ease the transition to C.

The problem is that moving one piece of your program to C isn't sufficient most of the time. Optimized Python programs usually don't have one major source of slowness, but rather, there are often many significant contributors. To get the benefits of C's bare metal and threads, you'd need to port large parts of your program, drastically increasing testing needs and risk. There must be a better way to preserve your investment in Python to solve difficult computational problems.

The `multiprocessing` built-in module, easily accessed via the `concurrent.futures` built-in module, may be exactly what you need. It enables Python to utilize multiple CPU cores in parallel by running additional interpreters as child processes. These child processes are separate from the main interpreter, so their global

interpreter locks are also separate. Each child can fully utilize one CPU core. Each child has a link to the main process where it receives instructions to do computation and returns results.

For example, say you want to do something computationally intensive with Python and utilize multiple CPU cores. I'll use an implementation of finding the greatest common divisor of two numbers as a proxy for a more computationally intense algorithm, like simulating fluid dynamics with the Navier-Stokes equation.

[Click here to view code image](#)

```
def gcd(pair):
    a, b = pair
    low = min(a, b)
    for i in range(low, 0, -1):
        if a % i == 0 and b % i == 0:
            return i
```

Running this function in serial takes a linearly increasing amount of time because there is no parallelism.

[Click here to view code image](#)

```
numbers = [(1963309, 2265973), (2030677, 3814172),
            (1551645, 2229620), (2039045, 2020802)]
start = time()
results = list(map(gcd, numbers))
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 1.170 seconds
```

Running this code on multiple Python threads will yield no speed improvement because the GIL prevents Python from using multiple CPU cores in parallel. Here, I do the same computation as above using the `concurrent.futures` module with its `ThreadPoolExecutor` class and two worker threads (to match the number of CPU cores on my computer):

[Click here to view code image](#)

```
start = time()
pool = ThreadPoolExecutor(max_workers=2)
results = list(pool.map(gcd, numbers))
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 1.199 seconds
```

It's even slower this time because of the overhead of starting and communicating with the pool of threads.

Now for the surprising part: By changing a single line of code, something magical happens. If I replace the `ThreadPoolExecutor` with the `ProcessPoolExecutor` from the `concurrent.futures` module, everything speeds up.

[Click here to view code image](#)

```
start = time()
```

```
pool = ProcessPoolExecutor(max_workers=2) # The one change
results = list(pool.map(gcd, numbers))
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 0.663 seconds
```

Running on my dual-core machine, it's significantly faster! How is this possible? Here's what the `ProcessPoolExecutor` class actually does (via the low-level constructs provided by the `multiprocessing` module):

1. It takes each item from the `numbers` input data to `map`.
2. It serializes it into binary data using the `pickle` module (see [Item 44: "Make pickle Reliable with copyreg"](#)).
3. It copies the serialized data from the main interpreter process to a child interpreter process over a local socket.
4. Next, it deserializes the data back into Python objects using `pickle` in the child process.
5. It then imports the Python module containing the `gcd` function.
6. It runs the function on the input data in parallel with other child processes.
7. It serializes the result back into bytes.
8. It copies those bytes back through the socket.
9. It deserializes the bytes back into Python objects in the parent process.
10. Finally, it merges the results from multiple children into a single list to return.

Although it looks simple to the programmer, the `multiprocessing` module and `ProcessPoolExecutor` class do a huge amount of work to make parallelism possible. In most other languages, the only touch point you need to coordinate two threads is a single lock or atomic operation. The overhead of using `multiprocessing` is high because of all of the serialization and deserialization that must happen between the parent and child processes.

This scheme is well suited to certain types of isolated, high-leverage tasks. By isolated, I mean functions that don't need to share state with other parts of the program. By high-leverage, I mean situations in which only a small amount of data must be transferred between the parent and child processes to enable a large amount of computation. The greatest common denominator algorithm is one example of this, but many other mathematical algorithms work similarly.

If your computation doesn't have these characteristics, then the overhead of `multiprocessing` may prevent it from speeding up your program through parallelization. When that happens, `multiprocessing` provides more advanced facilities for shared memory, cross-process locks, queues, and proxies. But all of these features are very complex. It's hard enough to reason about such tools in the memory space of a single process shared between Python threads. Extending that complexity to

other processes and involving sockets makes this much more difficult to understand.

I suggest avoiding all parts of `multiprocessing` and using these features via the simpler `concurrent.futures` module. You can start by using the `ThreadPoolExecutor` class to run isolated, high-leverage functions in threads. Later, you can move to the `ProcessPoolExecutor` to get a speedup. Finally, once you've completely exhausted the other options, you can consider using the `multiprocessing` module directly.

Things to Remember

- ◆ Moving CPU bottlenecks to C-extension modules can be an effective way to improve performance while maximizing your investment in Python code. However, the cost of doing so is high and may introduce bugs.
- ◆ The `multiprocessing` module provides powerful tools that can parallelize certain types of Python computation with minimal effort.
- ◆ The power of `multiprocessing` is best accessed through the `concurrent.futures` built-in module and its simple `ProcessPoolExecutor` class.
- ◆ The advanced parts of the `multiprocessing` module should be avoided because they are so complex.